Assignment #12
Self-Checking Testbenches

- **Self-Checking Testbench for a Multiplexer:**

  Design a 2:1 multiplexer and develop a self-checking testbench to verify all possible input combinations. Automatically compare the actual output with the expected output and report PASS or FAIL for each test.

- **Self-Checking Testbench for a Full Adder:**

  Design a full adder with inputs a, b, and c_{in}, and outputs sum and c_{out}. Verify all eight input combinations using an independent reference calculation in the testbench.

- **Self-Checking Testbench for a Comparator:**

  Design a 4-bit magnitude comparator with greater-than, less-than, and equality outputs. Verify boundary cases, equal operands, and operands with different magnitudes. Ensure that exactly one comparison output is asserted for every valid input pair.

- **Self-Checking Testbench for a 4-bit ALU:**

  Design a 4-bit ALU supporting addition, subtraction, bitwise AND, OR, XOR, left shift, right shift, and complement operations. Develop a reference model in the testbench and verify all operations with multiple operand combinations, including boundary values.

- **Self-Checking Testbench for a Priority Encoder:**

  Design an 8-to-3 priority encoder in which input bit $in[7]$ has the highest priority. Verify individual active inputs, multiple simultaneous active inputs, and the valid output signal.

- **Self-Checking Testbench for an Up/Down Counter:**

  Design a 4-bit synchronous up/down counter with reset and enable inputs. Verify increment, decrement, hold, reset, rollover from 15 to 0, and underflow from 0 to 15. Maintain an independent reference counter in the testbench and compare the DUT output after each active clock edge.

- **Self-Checking Testbench for a Sequence Detector:**

  Design an FSM-based sequence detector for the serial bit pattern 1011. Verify matching and non-matching sequences, consecutive detections, and overlapping occurrences. Check the detection output at the appropriate clock cycle.

- **Self-Checking Testbench for a Shift Register:**

  Design a 4-bit serial-in, parallel-out shift register with synchronous reset and enable. Verify shifting, holding the previous value when disabled, and reset operation. Compare the expected and actual register contents after every relevant clock edge.

- **Self-Checking Testbench for a Mod-10 Counter:**

  Design a synchronous counter that counts from 0 to 9 and then returns to 0. Verify all valid states, the transition from 9 to 0, and reset operation. Include a check to detect any unexpected or illegal state.

- **Self-Checking Testbench for a FIFO:**

  Design a synchronous FIFO with a depth of 4 and a data width of 8 bits. Verify write and read operations, full and empty flags, simultaneous read and write, attempts to write when full, and attempts to read when empty. Maintain a reference queue in the testbench and compare every valid read operation.

- **Self-Checking Testbench for a Parameterized Adder:**

  Design an N-bit adder using a Verilog parameter. Verify the design for at least two widths, such as 4 bits and 8 bits. Test normal additions, maximum operand values, and carry generation. Calculate the expected result independently in the testbench.

- **Self-Checking Testbench for a PWM Generator:**

  Design a pulse-width modulation generator with configurable period and duty cycle. Verify 0%, 25%, 50%, 75%, and 100% duty cycles. Count the high and low clock cycles and check the output period against the expected values.

- **Self-Checking Testbench for a Traffic Light Controller:**

  Design an FSM-based traffic light controller with red, yellow, and green outputs. Verify the correct state sequence, state durations, and reset operation. Add checks to detect illegal output combinations and incorrect state transitions.

- **Self-Checking Testbench for Signed Arithmetic:**

  Design an 8-bit signed arithmetic unit supporting addition, subtraction, and magnitude comparison. Verify positive and negative operands, zero, the boundary values -128 and $+127$, and signed overflow conditions. Use signed reference variables to calculate expected results.

- **Randomized Self-Checking Testbench for an ALU:**

  Reuse the 4-bit ALU from the previous exercise. Generate at least 100 pseudo-random input combinations and calculate the expected output using a reference model. For every mismatch, report the operation, operands, expected output, and actual output. Display the total number of passed and failed tests at the end of the simulation.

General Instructions

- Write the Verilog design module and its corresponding self-checking testbench for each assigned problem.
- Use conditional statements and comparison operators to compare the actual output with the expected output automatically.

- Use case inequality (`!==`) or case equality (`===`) when appropriate to detect unknown (`X`) and high-impedance (`Z`) values.
- For sequential circuits, verify outputs after the relevant clock edge and allow sufficient time for nonblocking assignments to take effect.
- Display a PASS or FAIL message for every test case. For failures, report the input conditions, expected result, and actual result.
- Maintain counters for the total number of successful and failed test cases and print a final summary at the end of simulation.
- Use an independent reference model to calculate expected results. Do not simply copy the DUT's implementation into the testbench.
- You may use Verilog file I/O system tasks, such as `$fopen`, `$fdisplay`, `$fwrite`, and `$fclose`, to read input test vectors from files and store simulation results, including PASS/FAIL status, expected outputs, actual outputs, and error details, in an output file. Note that file I/O system tasks are intended for simulation and are not synthesizable hardware constructs.
- Submit the Verilog source files and simulation logs or screenshots demonstrating the self-checking functionality.